

MLIR for AI/NPU Compiler

Lecture 01 Notes

MLIR 와 AI/NPU Compiler 큰 그림

Course	MLIR for AI and NPU Compiler
Lecture	01 - MLIR and AI/NPU Compiler Big Picture
Duration	90-120 minutes + lab
Output	NPU compiler lowering pipeline draft

1. Lecture Overview

1 강의 목적은 MLIR 문법을 모두 외우는 것이 아니라, AI 모델 그래프가 NPU 명령 스트림으로 내려가는 전체 compiler 지도를 먼저 만드는 것입니다. 이후 강의에서 각 단계의 문법, pass, lowering, runtime ABI 를 채워 넣습니다.

Key takeaway

MLIR 는 high-level model semantics 와 low-level NPU mapping 사이의 여러 abstraction level 을 dialect 와 pass pipeline 으로 연결합니다. NPU compiler 는 op legalization, fusion, tiling, bufferization, layout, DMA scheduling, command generation 을 검증 가능한 pipeline 으로 구성해야 합니다.

2. Learning Objectives

- AI/NPU compiler 의 end-to-end 역할을 설명할 수 있다.
- MLIR 가 multi-level IR 와 dialect ecosystem 을 통해 domain-specific compiler 를 구성하는 방식을 이해한다.
- StableHLO/TOSA -> Linalg/Vector -> MemRef/SCF -> Custom NPU Dialect 로 이어지는 lowering pipeline 을 그릴 수 있다.
- NPU 성능에 영향을 주는 compiler decision: tiling, fusion, layout, memory space, DMA schedule, quantization 을 설명할 수 있다.
- MatMul+Bias+ReLU 예제를 대상으로 NPU lowering pipeline 초안을 작성할 수 있다.

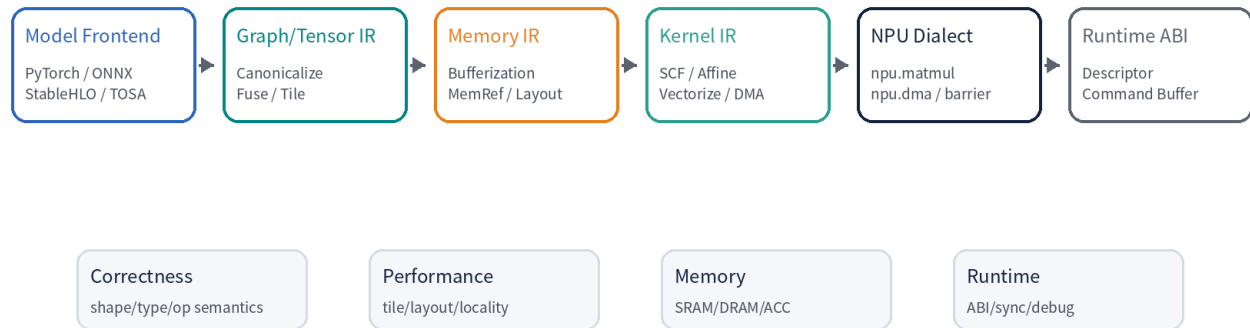
3. 왜 AI/NPU Compiler 에 MLIR 가 필요한가?

AI compiler 는 모델 그래프의 operator semantics, shape, broadcasting, quantization 정보와 NPU backend 의 PE array, SRAM, DMA, accumulator, command buffer ABI 를 동시에 다룹니다. 단일 IR 만으로는 high-level fusion 과 low-level memory scheduling 을 동시에 잘 표현하기 어렵습니다.

MLIR 의 핵심 가치는 각 단계에 맞는 dialect 를 사용하면서, pass pipeline 을 통해 점진적으로 lowering 하고 각 단계에서 verifier/test 를 걸 수 있다는 점입니다.

4. 전체 NPU Compiler Pipeline

MLIR-based AI/NPU Compiler Lowering Pipeline



Step	IR level	Purpose	NPU focus
Import	ONNX/StableHLO/TOSA	model semantics 확보	opset/version/shape validation
Legalize	Graph -> legal ops	unsupported op 분해	legal op coverage
Canonicalize	same-level IR	shape/algebra simplification	simplify before cost model
Fuse	Tensor/Linalg	DRAM traffic 감소	SRAM reuse
Tile	Tiled tensor/loop	SRAM/PE 에 맞는 tile 선택	PE utilization + SRAM footprint
Bufferize	Tensor -> MemRef	physical buffer/layout/space 결정	DRAM/SRAM/ACC mapping
Schedule	SCF/Affine/Vector/NPU	DMA/compute/barrier 순서	DMA/compute overlap
Emit	Runtime ABI	descriptor/command buffer 생성	runtime debugability

5. MLIR Core Concepts Preview

Operation: 계산/제어/선언의 기본 단위

Value: SSA data edge

Type: shape, element type, memory representation

Attribute: compile-time metadata: layout, quant, tile hint

Region/Block: operation 내부에 중첩되는 IR structure

Dialect: operation/type/attribute 의 namespace 와 semantic package

6. AI Model IR: StableHLO 와 TOSA

StableHLO 와 TOSA 는 NPU backend 가 바로 실행하는 instruction set 이라기보다, framework 와 backend 사이에서 model semantics 를 안정적으로 전달하기 위한 high-level/tensor-level IR 로 보는 것이 좋습니다.

IR	강점	NPU compiler 포인트
StableHLO	ML high-level op semantics 와 portability	dot_general, convolution, reduce 등 legalize
TOSA	DNN whole-tensor operation set	op coverage, quantized semantics

Custom NPU Dialect	target-specific op/memory/command	npu.matmul, npu.dma, npu.barrier, ABI
--------------------	-----------------------------------	---------------------------------------

7. NPU Target Model

- PE array shape 또는 tensor core tile shape
- On-chip SRAM 크기, bank 수, double buffering 가능 여부
- DMA granularity, alignment, burst efficiency
- Accumulator precision, supported element types, requantization 제한
- Runtime ABI: tensor descriptor, command buffer, synchronization model

Architect note

256+ TOPS 급 Edge NPU 에서도 실제 model latency 는 TOPS 만으로 결정되지 않습니다. compiler 가 DRAM traffic 을 줄이고 SRAM reuse 를 극대화하며 DMA/compute overlap 을 만들어야 체감 성능이 나옵니다.

8. Running Example: MatMul + Bias + ReLU

```
module {
  func.func @matmul_bias_relu(%A: tensor<1x128x256xf32>,
                              %B: tensor<256x512xf32>,
                              %bias: tensor<512xf32>)
    -> tensor<1x128x512xf32> {
    %0 = "stablehlo.dot_general"(%A, %B) : (...) -> tensor<1x128x512xf32>
    %1 = "stablehlo.add"(%0, %bias) : (...) -> tensor<1x128x512xf32>
    %2 = "stablehlo.maximum"(%1, %zero) : (...) -> tensor<1x128x512xf32>
    return %2 : tensor<1x128x512xf32>
  }
}
```

예상 lowering 방향: dot_general 과 add/maximum 을 fusion candidate 로 보고, M/N/K tile 을 정한 뒤, A/B/C tile 을 SRAM 에 배치하고, DMA load -> compute -> store 순서를 생성합니다.

9. Lab 1 설명

실습 목표는 주어진 MatMul+Bias+ReLU fragment 를 대상으로 컴파일러가 어떤 IR 단계와 pass 를 거쳐 NPU command 로 내려가는지 설계하는 것입니다.

가정: batch=1, M=128, K=256, N=512, PE output tile=64x64, K chunk=128, SRAM=2MB, DMA double-buffering 가능, FP16/FP32-acc 또는 INT8/INT32-acc 선택 가능.

제출 형식: pipeline table, target constraint table, dataflow sketch, correctness checks, performance metrics 를 1-2 페이지로 정리합니다.

10. Rubric

Criterion	Excellent answer
Pipeline completeness	Import/legalize/canonicalize/fuse/tile/bufferize/schedule/emit 포함
NPU awareness	Tile, SRAM, DMA, layout, accumulator constraints 연결
IR mapping	각 단계의 dialect 또는 IR level 명시
Correctness	Shape/type/semantic/golden/verifier checks 포함
Performance	Bandwidth, utilization, overlap, latency metrics 포함

11. References

- MLIR Language Reference: <https://mlir.llvm.org/docs/LangRef/>
- MLIR Understanding the IR Structure: <https://mlir.llvm.org/docs/Tutorials/UnderstandingTheIRStructure/>
- MLIR Using mlir-opt: <https://mlir.llvm.org/docs/Tutorials/MlirOpt/>
- MLIR Dialects: <https://mlir.llvm.org/docs/Dialects/>
- MLIR Bufferization: <https://mlir.llvm.org/docs/Bufferization/>
- StableHLO Specification: <https://openxla.org/stablehlo/spec>
- TOSA: <https://www.mlplatform.org/tosa/>
- MLIR TOSA Dialect: <https://mlir.llvm.org/docs/Dialects/TOSA/>